SmartBudget

Contents

Diagrams

The ER diagram is in Chen notation, using rectangles for entities, ovals for attributes with their SQL types, diamonds for relationships, and underlined primary keys. Every attribute includes its actual `CHECK`, `UNIQUE`, and `DEFAULT` clauses, while each relationship label specifies the FK/PK columns and the actual `ON DELETE` behavior. There are seven rectangles representing the seven tables; the schema contains no join table, so no M:N diamond appears and nothing has been collapsed. No weak entity is drawn because none exists, as both `budgets` and `ai_insights` have their own `INTEGER PRIMARY KEY AUTOINCREMENT` surrogate keys. The UML diagram does not attempt to map every class; instead, it shows only the classes involved in a design decision, organized into seven labelled clusters: Strategy, Adapter (+ Null Object), Observer, Decorator, Command, Singleton + Composition Root, and Simple Factory, with each cluster placed on a tinted backdrop and labelled with the corresponding pattern name. Every cluster also includes the client that triggers the pattern, since a pattern without its caller would not clearly demonstrate the design decision. Each cluster contains a note explaining why the pattern was necessary, which alternative was rejected and why, and one honest limitation of the implementation. Models, DAOs, and views that do not trigger any design pattern are deliberately omitted, as stated in the legend. Both diagrams are exported as `docs/ER.pdf` and `docs/UML.pdf`, allowing the repository to remain self-contained without requiring access to Lucid.

---

1. Architecture

```
UI (JavaFX views)
        ↓ calls
Service layer (business rules and validation)
        ↓ delegates to
Pattern components (Strategy, Observer, Decorator, Adapter, Factory, Command)
        ↓ reads/writes via
Persistence (Database + 6 DAOs)
        ↓ JDBC
SQLite (smartbudget.db)
```

The rule that holds it together: the UI never touches a DAO. Services receive their DAOs and strategies through constructors, so every service can be tested against an in-memory database with no JavaFX on the classpath. That is why 125 tests run in under two seconds and none of them opens a window.

`AppContext` builds the entire object graph in one readable method. It is deliberately explicit rather than a dependency-injection framework: the whole wiring of the application is visible in one place, and no view ever reaches for a singleton.

Layer boundaries

- `DataAccessException` wraps `SQLException` so `java.sql` never leaks past the persistence layer. Swapping SQLite for another store would not touch a service.
- `ValidationException` carries user-facing text and is caught at the view boundary in `UiSupport.guard`, so a blank description produces a dialog rather than a stack trace.

- ## 2. Database design

Six tables: `accounts`, `categories`, `transactions`, `budgets`, `recurring_rules`, `ai_insights`, plus a `schema_version` table used by the migration runner.

Money convention. `transactions.amount` is *signed*: negative is money out, positive is money in. The sign is the single source of truth for direction; the category is only a label. This is enforced with `CHECK (amount <> 0)` rather than left to convention.

Constraints that carry real weight, rather than being decorative:

Constraint	Why it exists
`CHECK (type IN ('checking','savings','credit'))`	The three account types behave differently; an invalid type would reach balance arithmetic
`CHECK (amount <> 0)`	A zero transaction is meaningless and would pollute averages
`CHECK (date IS strftime('%Y-%m-%d', date))`	Dates are compared as text; a malformed one sorts wrongly and silently disappears from month queries
`UNIQUE (category_id, month)` on budgets	Makes "set the budget" naturally an upsert instead of duplicate-prone
`CHECK ((related_transaction_id IS NULL) <> (related_report_month IS NULL))`	An insight explains exactly one subject — a transaction or a report month, never both
`ON DELETE CASCADE` on transactions	Deleting an account must not leave orphaned transactions
`ON DELETE SET NULL` on category	Deleting a category must not delete the user's spending history

`PRAGMA foreign_keys = ON` is set per connection, which is one of the reasons `Database` is a singleton (see below).

Seeding. `schema.sql` runs on every launch and is idempotent; `seed.sql` runs only when `schema_version` is empty, so a second launch never overwrites user data. Seed dates are computed relative to the current month (`date('now','start of month','-1 month')`), so sample data is always recent whenever the project is cloned and demonstrated.

3. Design patterns

Each pattern below answers the five questions asked in the demo: the problem, why this pattern, how it improved the design, what was rejected, and what it buys in future.

3.1 Strategy — categorisation

Problem. Categorisation has more than one plausible implementation: keyword and recurring-rule matching, a language model, and later possibly a local model. Encoding the choice as an `if` inside `TransactionService` would mean editing and re-testing that service every time a method was added, and would leave no way to substitute the AI path when the network is unavailable.

Why Strategy. The algorithm varies while the calling code does not. `TransactionService` depends only on `CategorizationStrategy`.

How it improved the design. Adding `AICategorizationStrategy` in Stage 5 introduced one new class and modified no existing logic — `TransactionService` was not touched. The AI strategy holds the rule-based strategy as its own fallback, so degradation is ordinary composition rather than a special case.

Alternative rejected. A single method with a mode flag. It keeps all branches in one class, which grows without bound and forces every caller to know which modes exist.

Future benefit. A local-model strategy, or a learned classifier trained on the user's own corrections, is one new class and one line in `AppContext`.

Where: `pattern/strategy/`, consumed by `service/TransactionService.java`.

3.2 Adapter — AI provider

Problem. Groq speaks an OpenAI-shaped protocol: nested `choices[0].message.content`, base64 image parts, bearer tokens, HTTP status codes. If categorisation, anomaly explanation and receipt import each spoke that protocol directly, changing provider would mean editing all three, and none could be tested without a network.

Why Adapter. It converts an external interface into the one this application actually wants. Callers ask for text and receive `Optional<String>`.

How it improved the design. Every AI feature is tested offline with a stub provider. The adapter's contract is that **it never throws** — no key, a timeout, a rate limit, malformed JSON, all become `Optional.empty()` — which makes graceful degradation a one-line `orElseGet` at each call site instead of try/catch repeated everywhere.

`NullAIProvider` is a **Null Object** rather than a `null` reference, so "AI not configured" and "AI call failed" travel the same code path. The fallback is therefore exercised constantly rather than only in a rare error case.

Alternative rejected. A shared HTTP utility method. That centralises the transport but still leaks the provider's JSON shape and error semantics into every caller, and leaves no seam to substitute in tests.

Future benefit. Supporting OpenAI, a local Ollama instance, or an offline model means one new implementation of `AIProvider`.

Where: `pattern/adapter/`.

3.3 Observer — budget alerts

Problem. Saving a transaction has to notify whatever currently cares about overspending — today the dashboard banner, later perhaps a notification tray or an email digest. Calling those directly from `BudgetService` would mean editing budget logic every time a consumer appeared, and would make the service impossible to unit test without constructing UI objects.

Why Observer. Publishers and subscribers vary independently. The budget rules answer "has a threshold been crossed"; who reacts is not their concern.

How it improved the design. `DashboardView` implements `BudgetListener` and subscribes to `BudgetEventBus`. Nothing in `BudgetService` knows a dashboard exists. `BudgetEventBus.publish` iterates over a snapshot of its listener list, so a listener may unsubscribe itself while being notified — a screen closing in response to an alert would otherwise throw `ConcurrentModificationException`.

The rule worth explaining in the demo: severity is computed *before and after* each transaction, and an alert is published only when it worsens. That is what makes the alert fire on the crossing itself rather than on every later purchase in an already-blown category, which would train the user to ignore it.

Alternative rejected. Passing a callback into each `BudgetService` call. It works for one consumer but degrades once several components need the same event, and pushes fan-out onto every caller.

Future benefit. A notification tray, a log file, or an email digest is one new listener and no change to any existing class.

Where: `pattern/observer/`, published by `service/BudgetService.java`.

3.4 Decorator — AI report insights

Problem. The monthly report needed an interpretation layer on top of the figures, and that layer is optional: the user can switch it off, and it must not exist at all when there is no network. Putting the narrative inside `BaseReport` would mean one class doing arithmetic and prose, with a boolean threaded through it.

Why Decorator. It adds behaviour to an object without changing that object's class or its interface. `BaseReport` was not modified, and callers cannot tell a wrapped report from a plain one.

How it improved the design. The Reports screen has a checkbox that swaps a decorated report for a plain one at runtime — the pattern is visible in a single click. Because `AIInsightReport` takes a `Report` rather than a `BaseReport`, decorators compose; there is a test that wraps a decorator in a decorator.

Alternative rejected. A subclass, `AIReport extends BaseReport`. That fixes the combination at compile time: wanting the AI layer over a quarterly report as well would need a second subclass, and every future pairing another.

Future benefit. A comparison-with-last-month layer, or a PDF-export layer, is another decorator that stacks on the existing ones.

Where: `pattern/decorator/`, toggled in `ui/ReportsView.java`.

3.5 Factory Method — account creation

Problem. The three account types do not start out alike. A credit card begins at zero debt and its balance means the *opposite* of a chequing balance, while deposit accounts begin from an opening balance. Left to the caller, every screen creating an account would repeat those rules and eventually get one wrong.

Why Factory Method. It puts type-specific construction in one place.

Honest scope note. This is the smallest pattern in the project and it is included because the types genuinely differ in initial state and validation — not merely in a label. Had the three types differed only by a string, a plain constructor would have been the better design and no pattern would appear here.

Alternative rejected. A plain constructor with defaults. It cannot express "a credit account may not open with a negative outstanding balance" without putting a conditional inside the model, mixing validation into a passive data holder.

Future benefit. A wallet or investment account type is a new branch in one method rather than a hunt for construction sites across the UI.

Where: `pattern/factory/AccountFactory.java`.

3.6 Command — natural-language entry with undo

Problem. Free-text entry turns one typed sentence into a validated database change. That change must be reversible, because a parser working from natural language will sometimes misread the amount, and the mistake is only obvious once the transaction appears in the table.

Why Command. It makes the requested action a first-class object, so it can be held, described and undone after the fact. A history of executed commands gives undo for free.

How it improved the design. `CommandHistory` owns the bookkeeping. `undo()` deletes through `TransactionService`, not the DAO, so the account balance is reversed by the same logic that applied it — deleting the row directly would leave the balance permanently wrong. A command that throws is never recorded, so a failed action can never be "undone" into an inconsistent state.

Alternative rejected. Having the parser call `TransactionService.create` directly. Simpler by one class, but there is then nothing to hold: undo would mean the UI separately remembering which row it just inserted, which is exactly the bookkeeping this pattern exists to own.

A note on justification. A Command with no `undo()` would be an ordinary method call wearing a class as a costume — the kind of decorative pattern use the project guidelines warn against. The undo is what earns it a place here.

Future benefit. A redo stack, or an audit log of every change, needs no change to callers.

Where: `pattern/command/`.

Patterns deliberately *not* used

- **Singleton** appears exactly once, for `Database`, and for a concrete reason: SQLite is a single file, `PRAGMA foreign_keys` is per-connection, and an in-memory test database only survives as long as its connection is held. Scattering `DriverManager.getConnection` through the DAOs would silently disable foreign keys on some of them. Crucially, the singleton is **not forced on the DAOs**: they receive a `Database` through their constructor, so tests inject `Database.inMemory()` and never call `getInstance()`.
- **Builder, Visitor, State, Template Method** were considered and rejected. No object here has enough optional construction parameters to justify a Builder, and nothing has a state machine complex enough to warrant State. Adding them would have been pattern-counting, not design.

4. Degradation without AI

Every AI feature has a working non-AI path. This is a design constraint, not a fallback of last resort.

Feature	With AI	Without AI
Categorisation	Model picks from the allowed list	Recurring rules, then ~35 keywords
Anomaly detection	**Statistical either way** — AI only writes the explanation	Explanation generated from the same figures
Subscription creep	**Statistical either way** — no AI involved in detecting	Identical
Monthly report insight	Model writes the commentary	Commentary assembled from the same figures
Natural-language entry	Model parses the sentence	Regex parser handles common phrasings
Receipt import	Vision model reads the photo	Not available — the form is filled in by hand

Only receipt import genuinely requires AI, and it says so plainly rather than failing silently.

Anomaly detection deserves emphasis. The decision is arithmetic — two standard deviations above the category mean, or three times the median, with a minimum sample of 4 and an amount floor of 500. The model is asked only for a sentence. Deciding in code rather than in the model means the feature works offline, is fully testable, and is grounded in this user's actual history rather than a model's guess about what "a lot" means.

5. Testing

125 tests, none requiring a network or a display.

Area	Tests	What is actually proven
Schema & DAOs	19	FK enforcement, CHECK rejection, cascade, data surviving close-and-reopen
Transaction service	16	Balance arithmetic for all three account types on create/update/delete
Budget service	13	Utilisation thresholds, and the alert firing **once** on the crossing
Patterns (Strategy/Factory/Observer)	18	Rule priority, keyword fallback, factory defaults, event dispatch
AI categorisation	9	Hallucinated category rejected, provider failure falls back, caching
Anomaly detection	9	Outliers, thin history, trivial amounts, income, scan idempotency
Subscription detection	12	Cadence, normalisation, both creep signals, user decisions respected
Receipt parsing	10	Prose and code fences, currency symbols, partial reads, bad dates
Decorator & Command	19	Wrapped sections preserved, decorators compose, undo restores balance

The tests worth demonstrating are the ones that would catch a real regression:

- `creditAndChequingMoveInOppositeDirections` — the balance inversion
- `alertFiresOnceOnCrossing` — the Observer's actual rule
- `undoRestores` — balance returns to exactly its previous value
- `decoratorPreservesAndAdds` — the wrapped report's sections are untouched
- `dataSurvivesRestartWithoutReseeding` — persistence, proven not assumed